COURS · PARTIE III

Web Sémantique

OWL : le langage d'ontologies du Web



2ème année Mastère Professionnel en Ingénierie des Systèmes d'Informations Décisionnels

Enseignante : Safa Chmengui

VUE D'ENSEMBLE

Plan du cours

1

Pourquoi OWL ?

Limites de RDFS, historique, structure d'un document

2

Les classes

Équivalence, disjonction, union, intersection, énumération

3

Les propriétés

Types, caractéristiques, inverse, chaînes de propriétés

4

Les restrictions

someValuesFrom, allValuesFrom, hasValue, cardinalités

5

Individus et monde ouvert

sameAs, differentFrom, hypothèse du monde ouvert

6

Raisonnement et pratique

Raisonneurs, profils OWL 2, Protégé, exercices

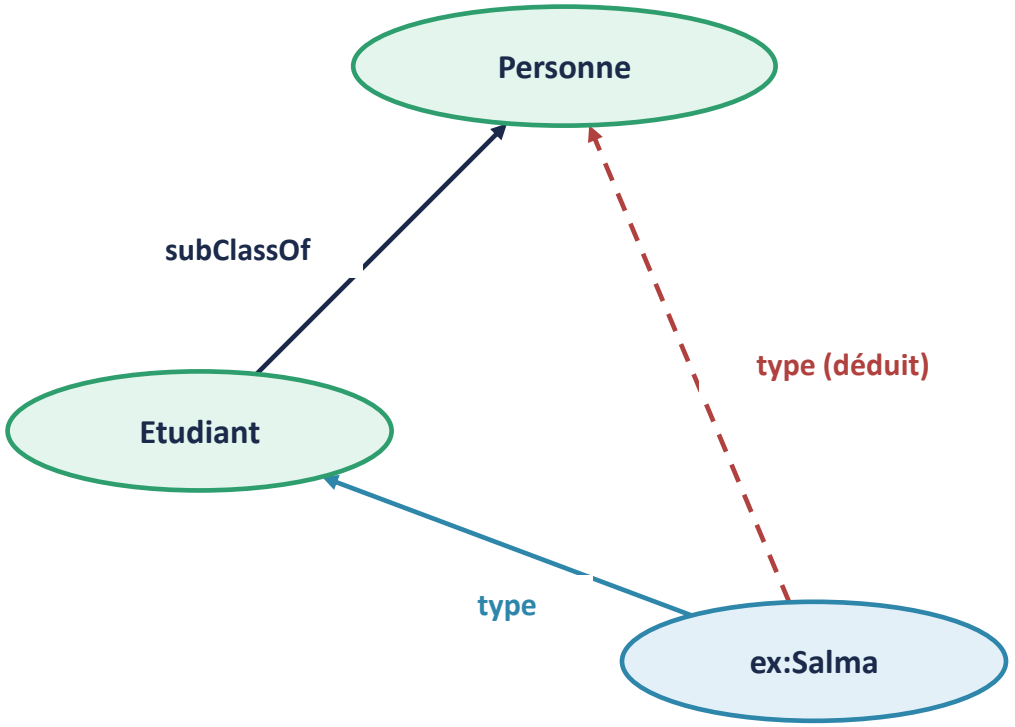
01

Pourquoi OWL ?

Les limites de RDFS et la naissance d'OWL

Rappel : ce que RDFS sait faire

- **Classes et hiérarchie** : rdfs:Class, rdfs:subClassOf (transitif)
- **Propriétés** : rdf:Property, rdfs:domain, rdfs:range, rdfs:subPropertyOf
- **Documentation** : rdfs:label, rdfs:comment
- **Inférence** : RDFS déduit de nouveaux triplets…
- **… mais n'interdit rien** : domain et range ne sont pas des contraintes



Fil rouge de ce cours : tout ce que RDFS ne peut pas dire, OWL le permet.

Les limites de RDFS (1/2) : on ne peut ni interdire, ni nier

Sami : Étudiant et Enseignant

On voudrait : interdire ce cas.

RDFS : accepte la donnée et déduit même que Sami est Enseignant (domain de « enseigne »).

Ali : Homme et Femme

On voudrait : dire que Homme et Femme n'ont aucun individu en commun.

RDFS : aucun moyen d'exprimer une disjonction.

Rami n'est pas Enseignant

On voudrait : une affirmation négative.

RDFS : n'exprime jamais de négation. Il répond « on ne sait pas ».

Ces trois cas viennent des exercices du cours précédent : RDFS déduit, mais il ne contraint pas et ne nie pas.

Les limites de RDFS (2/2) : ce qu'il ne sait pas exprimer

Besoin	Exemple	RDFS	OWL
Équivalence de classes	Enseignant = Formateur	contournement (cycle)	owl:equivalentClass
Disjonction	Étudiant ≠ Enseignant	impossible	owl:disjointWith
Combiner des classes	Parent = Mère ou Père	impossible	unionOf, intersectionOf
Propriétés spéciales	ancêtreDe est transitive	impossible	TransitiveProperty…
Propriété inverse	enseigne / estEnseignéPar	impossible	owl:inverseOf
Cardinalités	un cours a 1 seul enseignant	impossible	owl:cardinality…
Restrictions	Parent = a au moins un enfant	impossible	someValuesFrom…
Identité d'individus	Tunis (jeu A) = Tunis (jeu B)	impossible	owl:sameAs

OWL : définition et historique

2001 — **DAML+OIL**
fusion de DAML (USA) et OIL (Europe)

2004 — **OWL 1**
Recommandation W3C : Lite, DL, Full

2009 — **OWL 2**
Recommandation W3C : profils EL, QL, RL

2012 — **OWL 2, 2e édition**
version corrigée, aujourd'hui en usage

Web Ontology Language

Langage du W3C pour décrire des ontologies : classes, propriétés, individus et axiomes logiques.

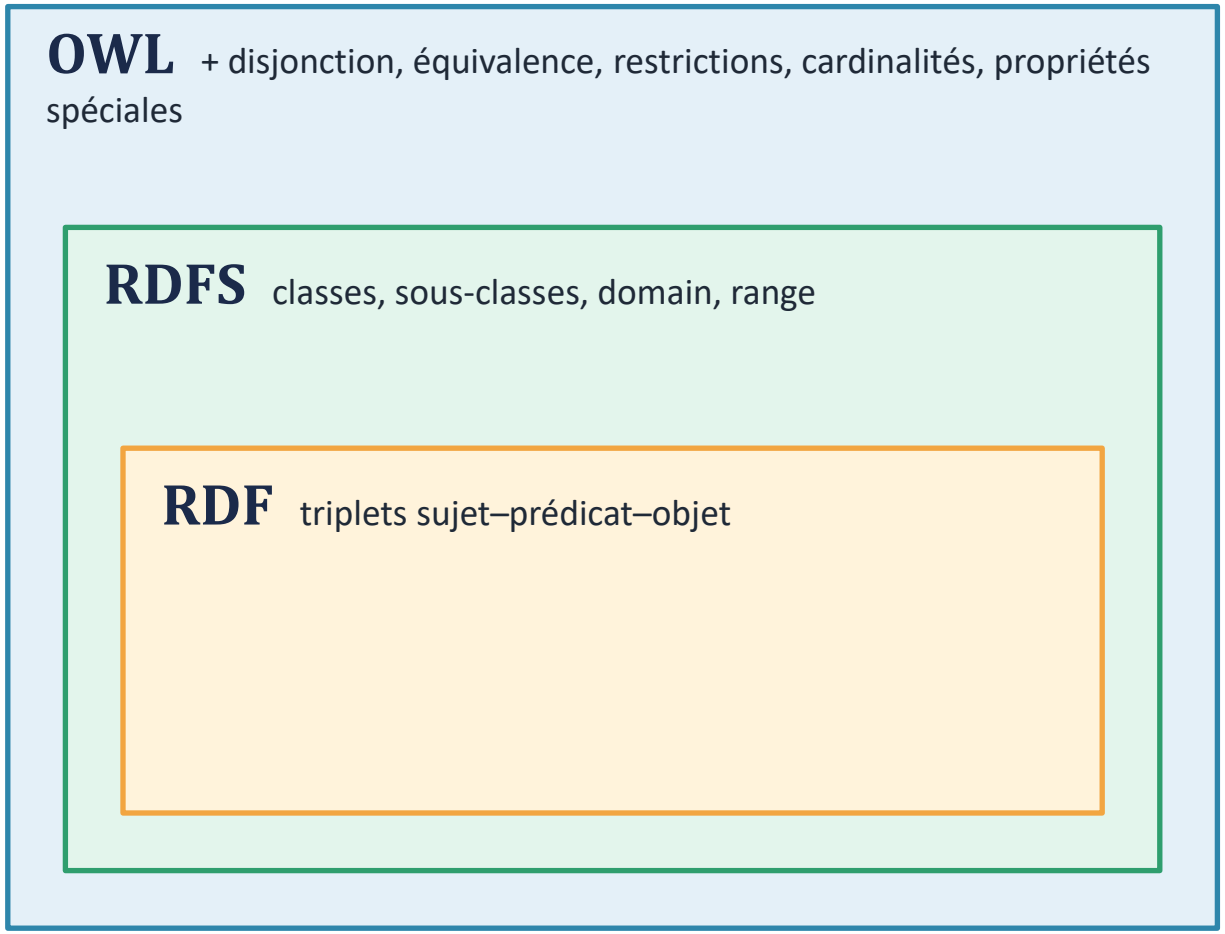
Fondé sur la logique

Basé sur les logiques de description : le sens est formel, donc un programme peut raisonner dessus.

Plus expressif que RDFS

Ajoute disjonction, équivalence, restrictions, cardinalités et propriétés spéciales.

OWL : une couche au-dessus de RDF et RDFS



- **OWL s'écrit en RDF** : un axiome OWL est un ensemble de triplets
- **OWL réutilise RDFS** : subClassOf, domain, range, label, comment
- **Syntaxes** : RDF/XML (celle du cours), Turtle, Manchester, Functional
- **Espaces de noms** : owl:, rdf:, rdfs:, xsd:

> Tout ce qui s'écrit en RDFS reste valable en OWL.

Structure d'un document OWL

```
<?xml version="1.0"?>
<rdf:RDF
  xmlns:rdf ="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:owl ="http://www.w3.org/2002/07/owl#"
  xmlns:xsd ="http://www.w3.org/2001/XMLSchema#"
  xmlns     ="http://exemple.tn/univ#"
  xml:base  ="http://exemple.tn/univ">

  <owl:Ontology rdf:about="">
    <rdfs:comment>Ontologie de l'université</rdfs:comment>
    <owl:versionInfo>1.0</owl:versionInfo>
  </owl:Ontology>

  <!-- classes, propriétés, individus ... -->
</rdf:RDF>
```

- **Racine** : rdf:RDF, comme en RDF/XML
- **Espaces de noms** : on ajoute owl: et xsd: à rdf: et rdfs:
- **xml:base** : URI de base de l'ontologie
- **owl:Ontology** : en-tête (commentaire, version, imports)

owl:imports permet de réutiliser une autre ontologie.

Une ontologie a deux niveaux : TBox et ABox

TBox : la terminologie (schéma)

Classes, propriétés et axiomes.

Etudiant ⊑ Personne
Etudiant et Enseignant sont disjoints
suit : Etudiant → Cours

ABox : les assertions (données)

Faits sur des individus précis.

Salma est un Etudiant
WebSemantique est un Cours
Salma suit WebSemantique

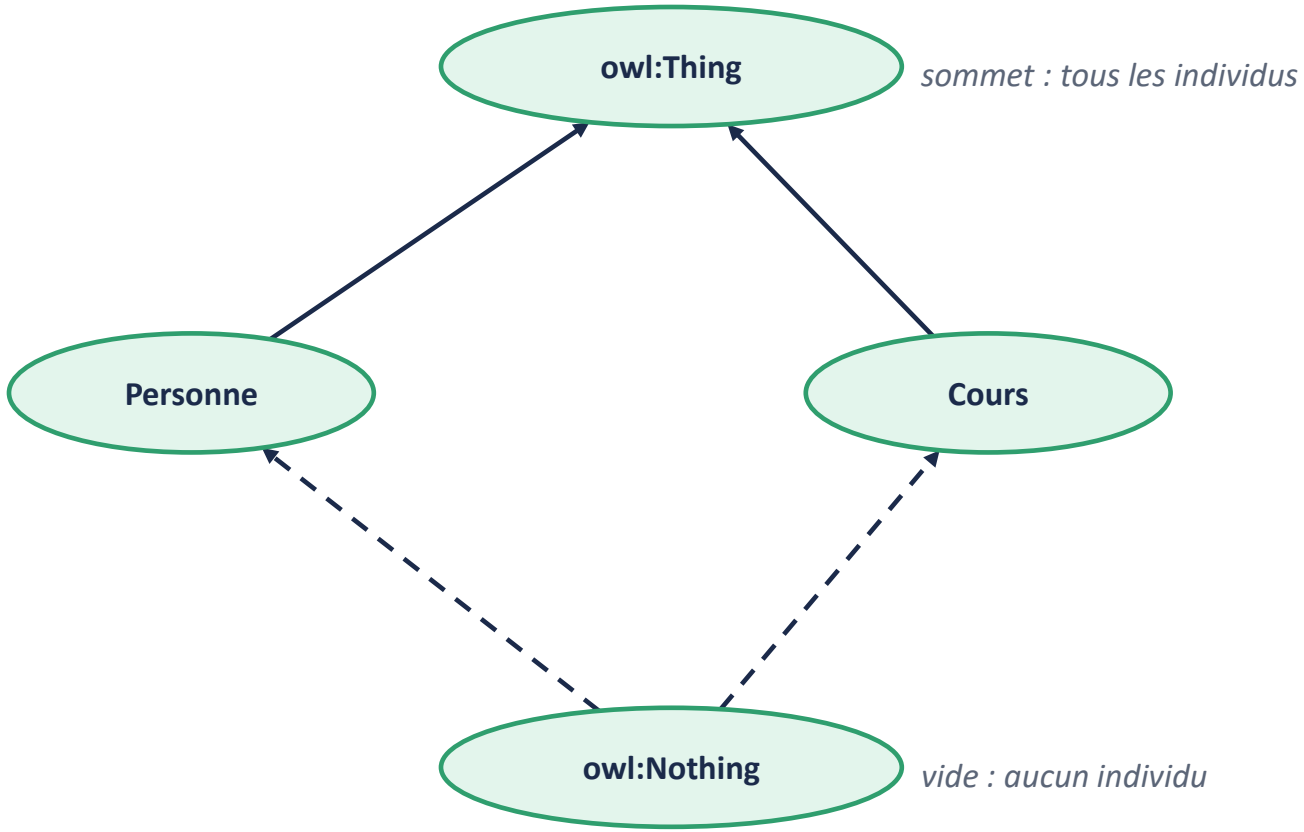
Le raisonneur combine TBox + ABox pour déduire de nouveaux faits. C'est la même idée que « schéma » et « données » du cours RDFS.

02

Les classes

Définir, relier et combiner des classes

owl:Class, owl:Thing et owl:Nothing



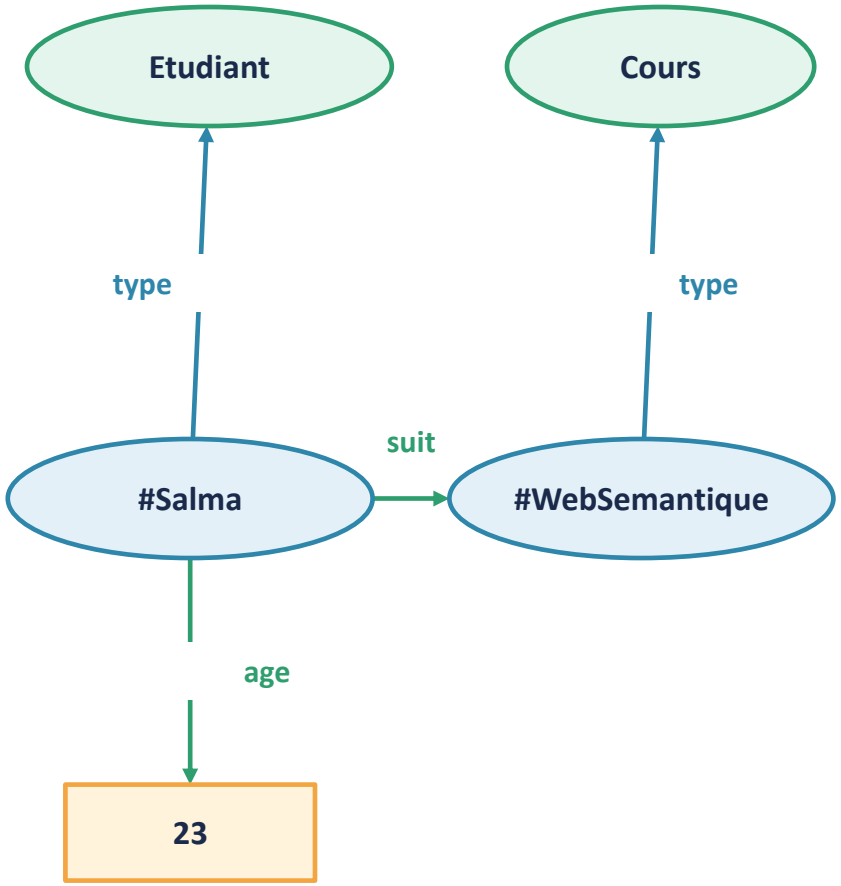
- **owl:Class** : classe OWL (spécialisation de rdfs:Class)
- **owl:Thing** : classe de tous les individus
- **owl:Nothing** : classe vide
- Une classe équivalente à owl:Nothing est insatisfiable : c'est une erreur de modélisation

```
<owl:Class rdf:ID="Personne"/>
<owl:Class rdf:ID="Etudiant">
  <rdfs:subClassOf rdf:resource="#Personne"/>
</owl:Class>
```

Les individus : owl:NamedIndividual

```
<owl:NamedIndividual rdf:about="#Salma">
  <rdf:type rdf:resource="#Etudiant"/>
  <suit rdf:resource="#WebSemantique"/>
  <age rdf:datatype=
       "http://www.w3.org/2001/XMLSchema#integer">23</age>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#WebSemantique">
  <rdf:type rdf:resource="#Cours"/>
</owl:NamedIndividual>
```

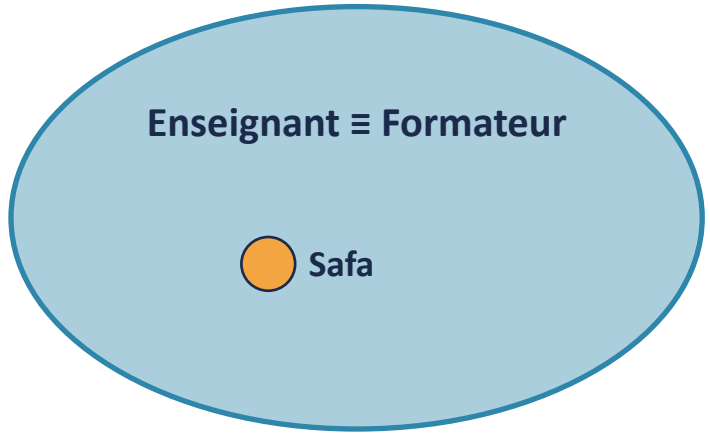


- **Un individu est une instance d'une classe** (rdf:type)
- **Ses propriétés sont ses assertions** : lien vers une ressource ou une valeur typée

Équivalence de classes : owl:equivalentClass

```
<owl:Class rdf:ID="Enseignant">
  <owl:equivalentClass rdf:resource="#Formateur"/>
</owl:Class>

<owl:NamedIndividual rdf:about="#Safa">
  <rdf:type rdf:resource="#Enseignant"/>
</owl:NamedIndividual>
```



- **Deux classes équivalentes ont exactement les mêmes individus**
- **Plus fort que subClassOf** : l'inclusion marche dans les deux sens
- En RDFS, on devait faire un cycle de subClassOf. OWL le dit directement.

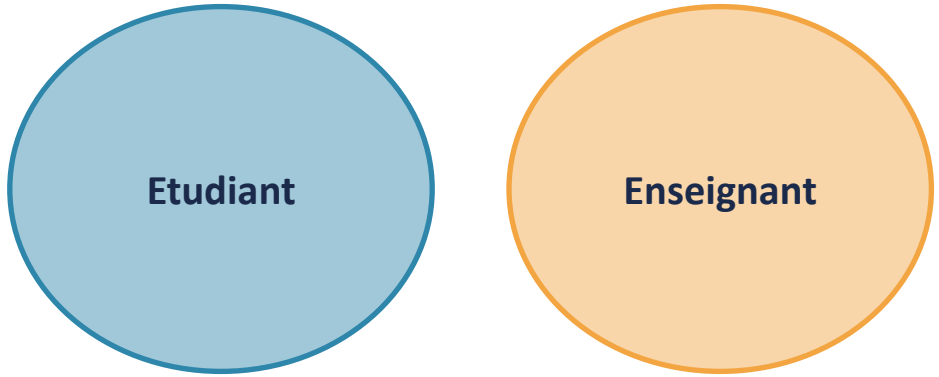
Déductions du raisonneur

```
Safa type Formateur
tout Formateur est aussi Enseignant
```

Classes disjointes : owl:disjointWith

```
<owl:Class rdf:about="#Etudiant">
  <owl:disjointWith rdf:resource="#Enseignant"/>
</owl:Class>

<!-- plusieurs classes deux à deux disjointes -->
<owl:AllDisjointClasses>
  <owl:members rdf:parseType="Collection">
    <owl:Class rdf:about="#Etudiant"/>
    <owl:Class rdf:about="#Enseignant"/>
    <owl:Class rdf:about="#Personnel"/>
  </owl:members>
</owl:AllDisjointClasses>
```



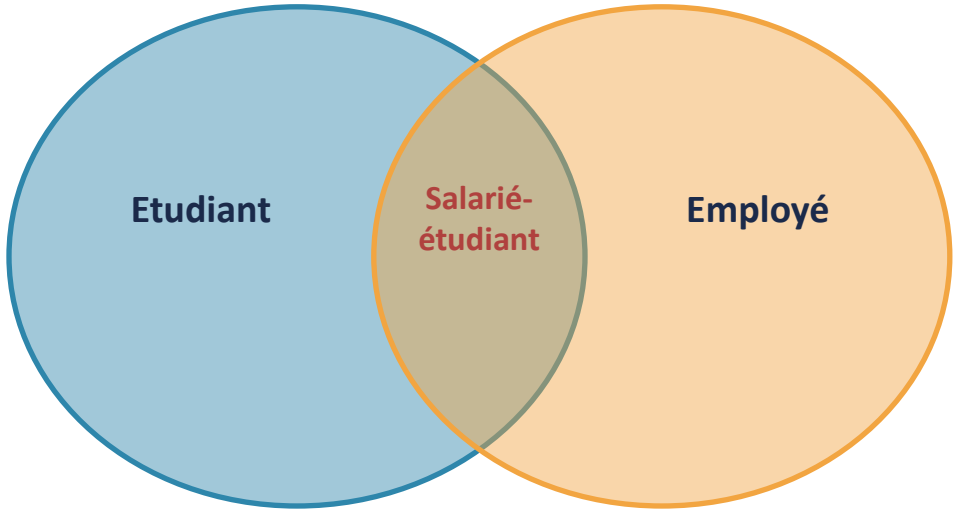
aucun individu en commun

C'est la réponse à l'exercice « Sami » : en OWL, on peut enfin interdire ce cas.

Incohérence détectée

`Sami type Etudiant`
`Sami type Enseignant`
→ Sami serait dans deux classes disjointes : le raisonneur signale une erreur.

Opérations sur les classes (1/2) : intersectionOf



EtudiantSalarié ≡ Etudiant ∩ Employé

Intersection = « à la fois … et … »

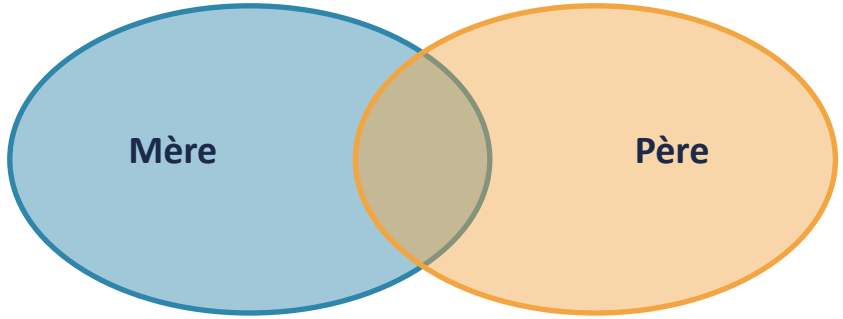
```
<owl:Class rdf:ID="EtudiantSalarie">
  <owl:equivalentClass>
    <owl:Class>
      <owl:intersectionOf rdf:parseType="Collection">
        <owl:Class rdf:about="#Etudiant"/>
        <owl:Class rdf:about="#Employe"/>
      </owl:intersectionOf>
    </owl:Class>
  </owl:equivalentClass>
</owl:Class>
```

Déductions du raisonneur

```
Nour type Etudiant
Nour type Employe
=> Nour type EtudiantSalarie
```

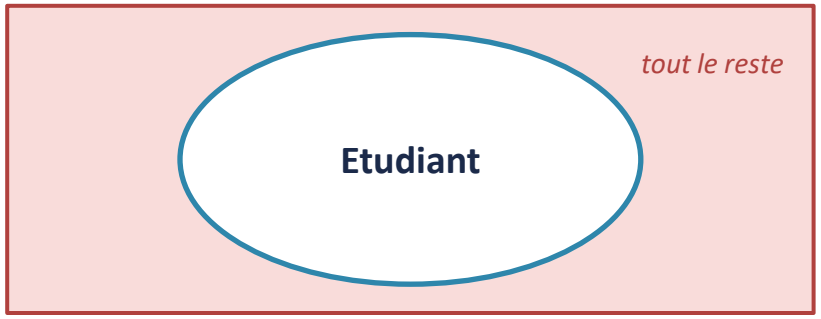
Opérations sur les classes (2/2) : unionOf et complementOf

Union : Parent ≡ Mère ∪ Père




```
<owl:Class rdf:ID="Parent">
  <owl:equivalentClass><owl:Class>
    <owl:unionOf rdf:parseType="Collection">
      <owl:Class rdf:about="#Mere"/>
      <owl:Class rdf:about="#Pere"/>
    </owl:unionOf>
  </owl:Class></owl:equivalentClass>
</owl:Class>
```

Complément : NonEtudiant ≡ ¬ Etudiant



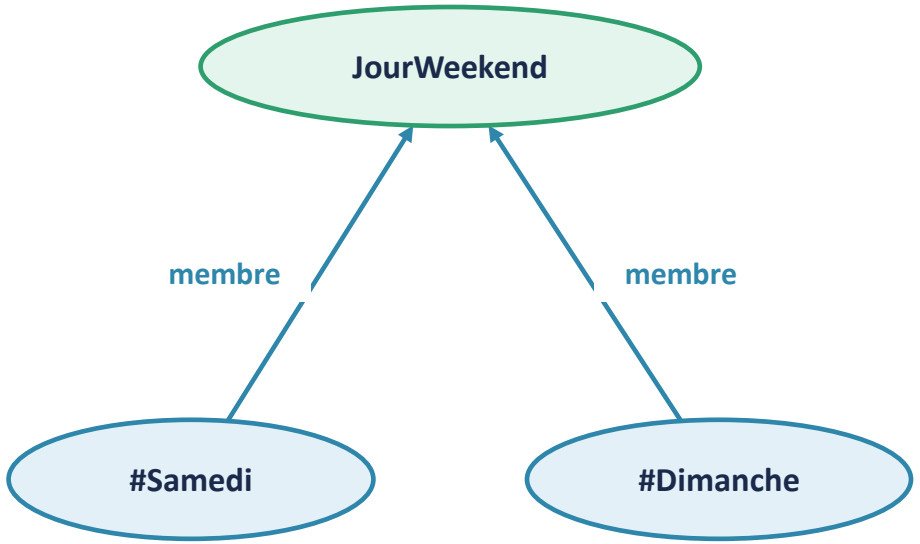

```
<owl:Class rdf:ID="NonEtudiant">
  <owl:equivalentClass><owl:Class>
    <owl:complementOf rdf:resource="#Etudiant"/>
  </owl:Class></owl:equivalentClass>
</owl:Class>
```

Union = « ou ». Complément = « tout sauf ». Attention : complément ≠ disjonction (le complément couvre tout le reste).

Classes énumérées : owl:oneOf

```
<owl:Class rdf:ID="JourWeekend">
  <owl:oneOf rdf:parseType="Collection">
    <rdf:Description rdf:about="#Samedi"/>
    <rdf:Description rdf:about="#Dimanche"/>
  </owl:oneOf>
</owl:Class>
```

- **On définit une classe en listant ses membres**
- La classe est fermée : il n'y en a pas d'autres
- Utile pour les listes de valeurs fixes : jours, niveaux (L, M, D), statuts



Déductions du raisonneur

```
Si X type JourWeekend
=> X est #Samedi ou #Dimanche
```

03

Les propriétés

Types, caractéristiques, inverse et chaînes

Trois types de propriétés en OWL

owl:ObjectProperty

Relie deux ressources.
L'objet est une ressource (bulle).

ex : enseigne, habiteEn, suit

owl:DatatypeProperty

Relie une ressource à une valeur typée (xsd).
L'objet est un littéral (rectangle).

ex : age, email, titre

owl:AnnotationProperty

Documente, sans effet sur le raisonnement.

ex : rdfs:label, rdfs:comment

```
<owl:ObjectProperty   rdf:ID="enseigne"/>
<owl:DatatypeProperty rdf:ID="age"/>
<owl:AnnotationProperty rdf:about="http://www.w3.org/2000/01/rdf-schema#label"/>
```

C'est la distinction « l'objet est une ressource ou un littéral », maintenant écrite dans le schéma.

domain, range et types de données (XSD)

```
<owl:ObjectProperty rdf:ID="enseigne">
  <rdfs:domain rdf:resource="#Enseignant"/>
  <rdfs:range  rdf:resource="#Cours"/>
</owl:ObjectProperty>

<owl:DatatypeProperty rdf:ID="age">
  <rdfs:domain rdf:resource="#Personne"/>
  <rdfs:range  rdf:resource=
    "http://www.w3.org/2001/XMLSchema#integer"/>
</owl:DatatypeProperty>
```

Type XSD	Exemple de valeur
xsd:string	"Safa"
xsd:integer	23
xsd:decimal	14.75
xsd:boolean	true
xsd:date	2026-10-17
xsd:dateTime	2026-10-17T14:30:00

Comme en RDFS, domain et range servent à déduire des types. Ils deviennent bloquants seulement quand on les combine avec des classes disjointes.

Caractéristiques des propriétés : vue d'ensemble

Caractéristique	Signification	Exemple	Effet
Functional	au plus une valeur par sujet	aPourMère	deux valeurs ⇒ identiques
InverseFunctional	une valeur identifie un seul sujet	numCIN, email	même valeur ⇒ même sujet
Transitive	A→B et B→C ⇒ A→C	ancêtreDe	chaîne complétée
Symmetric	A→B ⇒ B→A	estMarié	sens inverse ajouté
Asymmetric	A→B interdit B→A	parentDe	les deux sens ⇒ erreur
Reflexive / Irreflexive	chacun est lié (ou jamais) à lui-même	estProcheDe / parentDe	A→A déduit / interdit

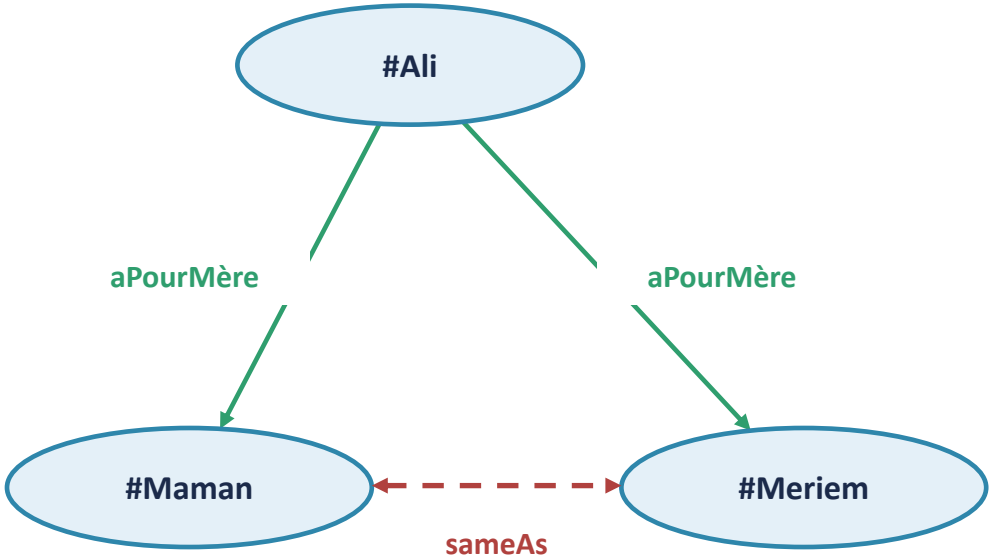
Chaque caractéristique est une classe OWL qu'on ajoute avec rdf:type sur la propriété.

owl:FunctionalProperty : au plus une valeur

```
<owl:ObjectProperty rdf:ID="aPourMere">
  <rdf:type rdf:resource=
    "http://www.w3.org/2002/07/owl#FunctionalProperty"/>
</owl:ObjectProperty>

<owl:NamedIndividual rdf:about="#Ali">
  <aPourMere rdf:resource="#Maman"/>
  <aPourMere rdf:resource="#Meriem"/>
</owl:NamedIndividual>
```

- **Une seule mère possible** : le raisonneur en déduit que Maman et Meriem sont le même individu
- **Si on a déclaré Maman differentFrom Meriem** : incohérence



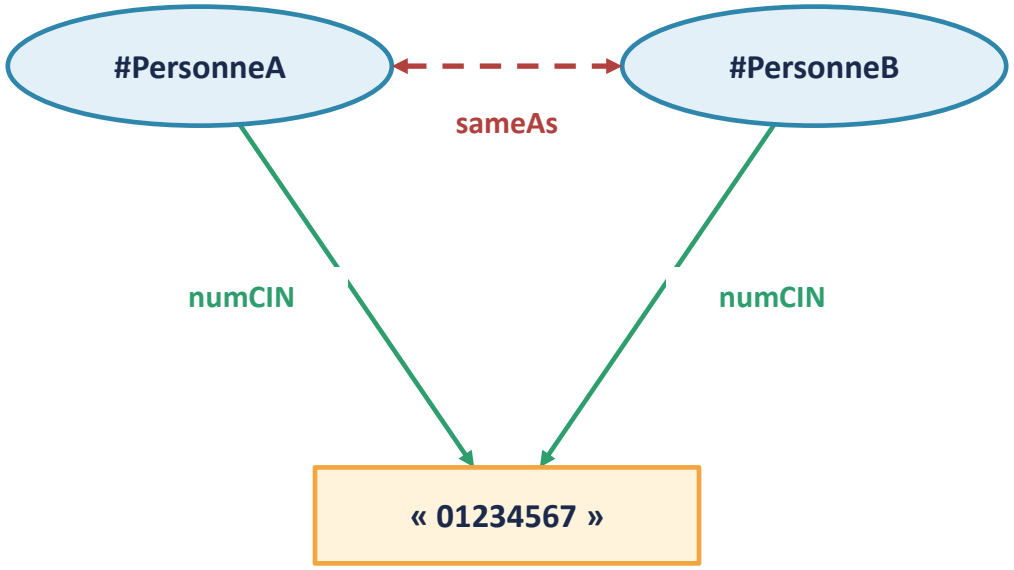
Déductions du raisonneur

```
#Maman owl:sameAs #Meriem
```

owl:InverseFunctionalProperty : valeur = identifiant

```
<owl:DatatypeProperty rdf:ID="numCIN">
  <rdf:type rdf:resource=
    "http://www.w3.org/2002/07/owl#InverseFunctionalProperty"/>
</owl:DatatypeProperty>

<rdf:Description rdf:about="#PersonneA">
  <numCIN>01234567</numCIN>
</rdf:Description>
<rdf:Description rdf:about="#PersonneB">
  <numCIN>01234567</numCIN>
</rdf:Description>
```



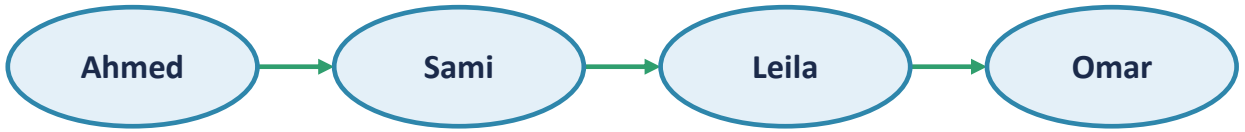
- **Même valeur ⇒ même individu** : comme une clé primaire en base de données
- **Exemples** : numéro de CIN, email, ISBN, matricule

Functional : sujet → une valeur. InverseFunctional : valeur → un sujet.

owl:TransitiveProperty : compléter les chaînes

```
<owl:ObjectProperty rdf:ID="ancetreDe">
  <rdf:type rdf:resource=
    "http://www.w3.org/2002/07/owl#TransitiveProperty"/>
</owl:ObjectProperty>

<!-- Ahmed ancetreDe Sami -->
<!-- Sami  ancetreDe Leila -->
<!-- Leila ancetreDe Omar  -->
```



ancêtreDe (déclarés)

- **Règle** : A ancêtreDe B et B ancêtreDe C ⇒ A ancêtreDe C
- **Le raisonneur ajoute les liens manquants** sans qu'on les écrive

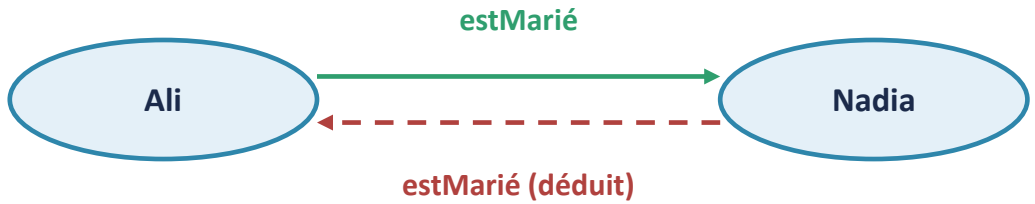
Déductions du raisonneur

```
Ahmed ancetreDe Leila
Ahmed ancetreDe Omar
Sami  ancetreDe Omar
```

OWL DL : on ne met pas de cardinalité sur une propriété transitive.

Propriétés symétrique et asymétrique

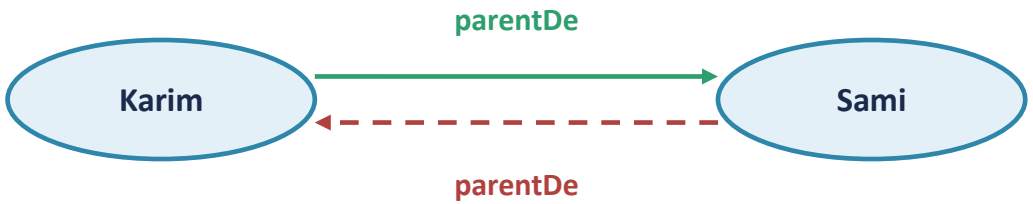
owl:SymmetricProperty




```
<owl:ObjectProperty rdf:ID="estMarie">
  <rdf:type rdf:resource=
    "http://www.w3.org/2002/07/owl#SymmetricProperty"/>
</owl:ObjectProperty>
```

Ali estMarié Nadia ⇒ Nadia estMarié Ali

owl:AsymmetricProperty




```
<owl:ObjectProperty rdf:ID="parentDe">
  <rdf:type rdf:resource=
    "http://www.w3.org/2002/07/owl#AsymmetricProperty"/>
</owl:ObjectProperty>
```

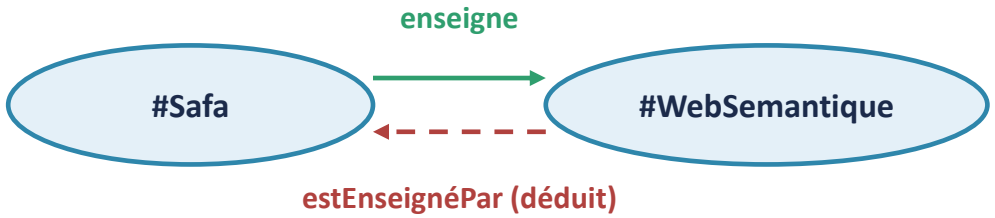
Les deux sens à la fois ⇒ incohérence

Symétrique : le sens inverse est ajouté. Asymétrique : le sens inverse est interdit.

Propriété inverse : owl:inverseOf

```
<owl:ObjectProperty rdf:ID="enseigne">
  <owl:inverseOf rdf:resource="#estEnseignePar"/>
</owl:ObjectProperty>

<owl:NamedIndividual rdf:about="#Safa">
  <enseigne rdf:resource="#WebSemantique"/>
</owl:NamedIndividual>
```



- **Deux propriétés qui disent la même chose dans le sens opposé**
- On écrit un seul sens ; l'autre est déduit
- domain et range s'échangent : domain(enseigne) = range(estEnseignéPar)

Déductions du raisonneur

```
#WebSemantique estEnseignePar #Safa
```

Utile pour interroger dans les deux directions sans dupliquer les données.

Équivalence et hiérarchie de propriétés

owl:equivalentProperty

```
<owl:ObjectProperty rdf:ID="habiteEn">
  <owl:equivalentProperty
      rdf:resource="#resideDans"/>
</owl:ObjectProperty>
```

Mohamed habiteEn Tunis ⇒ Mohamed resideDans Tunis

Utile pour aligner deux vocabulaires qui nomment la même relation différemment.

rdfs:subPropertyOf

```
<owl:ObjectProperty rdf:ID="dirige">
  <rdfs:subPropertyOf
      rdf:resource="#travailleDans"/>
</owl:ObjectProperty>
```

Nour dirige Finance ⇒ Nour travailleDans Finance

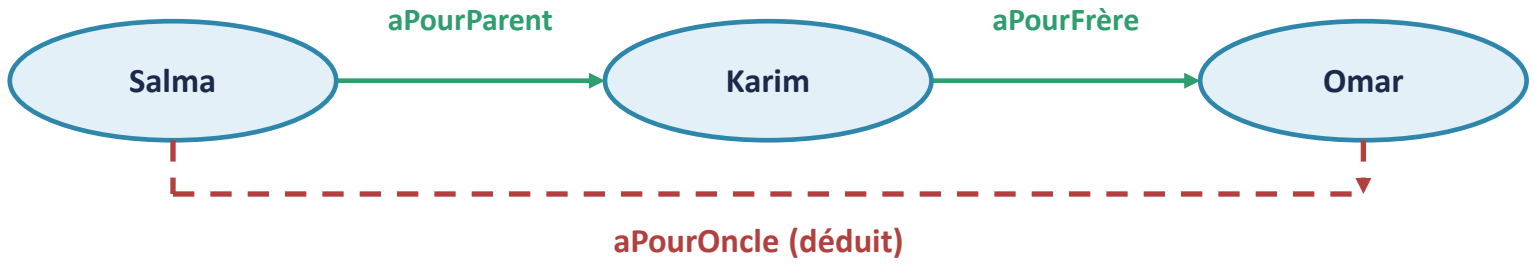
Vu dans les exercices : la propriété la plus précise implique la plus générale.

equivalentProperty = inclusion dans les deux sens. subPropertyOf = inclusion dans un seul sens.

Chaîne de propriétés : owl:propertyChainAxiom

```
<owl:ObjectProperty rdf:ID="aPourOncle">
  <owl:propertyChainAxiom rdf:parseType="Collection">
    <owl:ObjectProperty rdf:about="#aPourParent"/>
    <owl:ObjectProperty rdf:about="#aPourFrere"/>
  </owl:propertyChainAxiom>
</owl:ObjectProperty>
```

- **Compose deux propriétés pour en déduire une troisième**
- **Lecture** : le parent de Salma est Karim, le frère de Karim est Omar, donc Omar est l'oncle de Salma



> Les chaînes expriment des règles que ni RDFS ni les sous-propriétés ne peuvent écrire.

04

Les restrictions

Définir une classe par ce que ses membres vérifient

Restrictions : définir une classe par ses propriétés

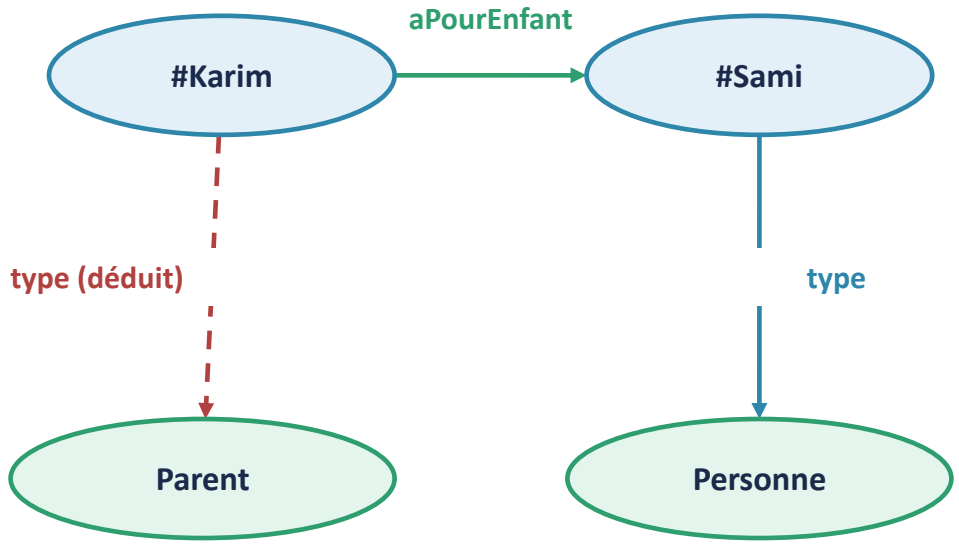
```
<owl:Restriction>
  <owl:onProperty rdf:resource="#p"/>
  <owl:someValuesFrom rdf:resource="#C"/>
</owl:Restriction>
```

- **Une restriction est une classe anonyme** : l'ensemble des individus qui vérifient une condition sur une propriété
- **Structure** : owl:onProperty + une condition
- **On l'utilise dans subClassOf ou equivalentClass**

Condition	Notation	Sens	Exemple
someValuesFrom	∃ p.C	au moins une valeur dans C	a au moins un enfant
allValuesFrom	∀ p.C	toutes les valeurs dans C	ne mange que du végétal
hasValue	p = v	a pour valeur précise v	habite Tunis
Cardinalités	≥ n, ≤ n, = n	nombre de valeurs	exactement 1 responsable

owl:someValuesFrom (∃) : « au moins une valeur de »

```
<owl:Class rdf:ID="Parent">
  <owl:equivalentClass>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#aPourEnfant"/>
      <owl:someValuesFrom rdf:resource="#Personne"/>
    </owl:Restriction>
  </owl:equivalentClass>
</owl:Class>
```



Parent ≡ ∃ aPourEnfant . Personne

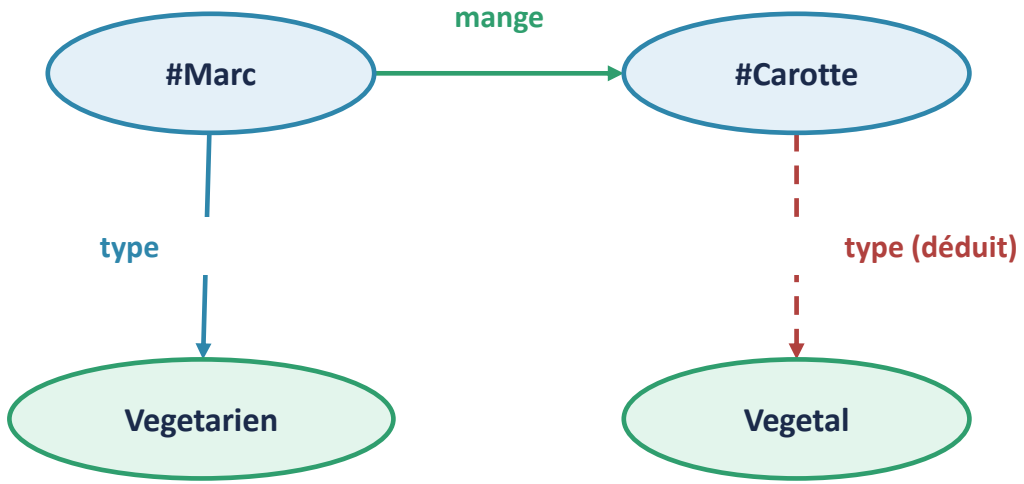
- Un Parent est une personne qui a au moins un enfant qui est une Personne
- equivalentClass : condition nécessaire et suffisante, donc classement automatique

Déductions du raisonneur

```
Karim type Parent
```

owl:allValuesFrom (∀) : « uniquement des valeurs de »

```
<owl:Class rdf:ID="Vegetarien">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#mange"/>
      <owl:allValuesFrom rdf:resource="#Vegetal"/>
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```



Végétarien ⊑ ∀ mange . Végétal

Piège 1 : vérité à vide

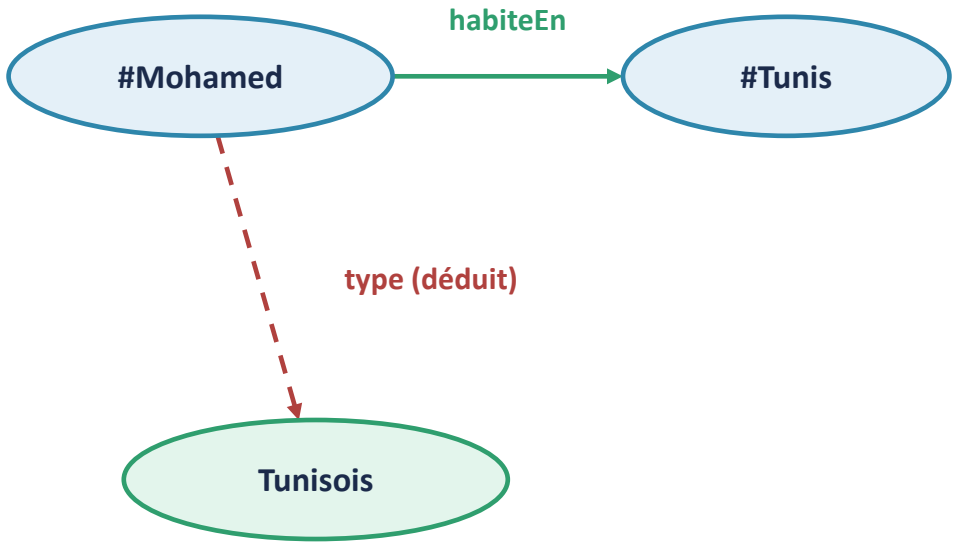
Quelqu'un qui ne mange rien vérifie « ∀ mange.Végétal ». ∀ ne garantit pas qu'il existe une valeur.

Piège 2 : monde ouvert

Le raisonneur ne conclut pas « Marc n'est pas végétarien » parce qu'il n'a vu aucun légume.

owl:hasValue : « a pour valeur précise »

```
<owl:Class rdf:ID="Tunisois">
  <owl:equivalentClass>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#habiteEn"/>
      <owl:hasValue rdf:resource="#Tunis"/>
    </owl:Restriction>
  </owl:equivalentClass>
</owl:Class>
```



Tunisois ≡ habiteEn = Tunis

- **hasValue fixe une valeur unique** : un individu ou un littéral
- **Différence avec someValuesFrom** : ici la valeur est un individu précis, pas une classe

Déductions du raisonneur

```
Mohamed type Tunisois
```

Cardinalités : minCardinality, maxCardinality, cardinality

Élément OWL	Sens	Notation	Exemple
owl:minCardinality	au moins n valeurs	≥ n p	MasterPro : au moins 4 modules
owl:maxCardinality	au plus n valeurs	≤ n p	Cours : au plus 1 responsable
owl:cardinality	exactement n valeurs	= n p	Cours : exactement 1 responsable

```
<owl:Class rdf:about="#Cours">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#aPourResponsable"/>
      <owl:cardinality rdf:datatype=
        "http://www.w3.org/2001/XMLSchema#nonNegativeInteger">1</owl:cardinality>
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

> Les cardinalités comptent des individus distincts. Or OWL ne suppose pas que deux noms désignent deux individus : voir la diapo suivante.

- **n est un entier positif ou nul** (xsd:nonNegativeInteger)
- **OWL 2 ajoute les cardinalités qualifiées** : owl:minQualifiedCardinality + owl:onClass (« au moins 2 enseignants de la classe Docteur »)

Cardinalité et absence d'hypothèse du nom unique

Règle : un Cours a exactement 1 responsable. On déclare : C1 aPourResponsable Karim et C1 aPourResponsable Leila

Cas A : rien de plus



Le raisonneur déduit que Karim et Leila sont le même individu. Pas d'erreur.

Cas B : Karim differentFrom Leila



Les deux sont distincts mais le cours n'a qu'un responsable : incohérence détectée.

Pour que OWL détecte l'erreur, il faut déclarer explicitement que les individus sont différents.

Condition nécessaire ou nécessaire et suffisante

rdfs:subClassOf : condition nécessaire

$Parent \sqsubseteq \exists\ aPourEnfant\ .\ Personne$

- Tout Parent a au moins un enfant
- Mais avoir un enfant ne suffit pas à être classé Parent

`Karim aPourEnfant Sami`

Karim est un Parent ? Non, pas déduit

owl:equivalentClass : nécessaire et suffisante

$Parent \equiv \exists\ aPourEnfant\ .\ Personne$

- Tout Parent a au moins un enfant
- Et tout individu qui a un enfant est un Parent

`Karim aPourEnfant Sami`

Karim est un Parent ? Oui, déduit

Pour classer automatiquement des individus, il faut equivalentClass. subClassOf seul ne le permet pas.

05

Individus et monde ouvert

Identité, hypothèse du monde ouvert et fermeture

Identité : owl:sameAs, owl:differentFrom, owl:AllDifferent

```
<rdf:Description rdf:about="http://a.org/Tunis">
  <owl:sameAs rdf:resource="http://b.org/Tunis"/>
</rdf:Description>

<rdf:Description rdf:about="#Karim">
  <owl:differentFrom rdf:resource="#Leila"/>
</rdf:Description>

<owl:AllDifferent>
  <owl:distinctMembers rdf:parseType="Collection">
    <rdf:Description rdf:about="#Karim"/>
    <rdf:Description rdf:about="#Leila"/>
    <rdf:Description rdf:about="#Omar"/>
  </owl:distinctMembers>
</owl:AllDifferent>
```

- **sameAs** : deux URI désignent le même individu (lier des jeux de données)
- **differentFrom** : deux individus distincts
- **AllDifferent** : tous distincts deux à deux

OWL n'a pas l'hypothèse du nom unique : deux noms différents peuvent désigner le même individu. Une base de données suppose le contraire.

Hypothèse du monde ouvert (OWA) et du monde fermé (CWA)

Base de données : monde fermé

Ce qui n'est pas enregistré est faux.

```
Faits : Rami suit Algo
```

Rami suit-il Réseaux ?

Réponse : NON

OWL : monde ouvert

Ce qui n'est pas dit est inconnu.

```
Faits : Rami suit Algo
```

Rami suit-il Réseaux ?

Réponse : ON NE SAIT PAS

- **Le Web est ouvert** : d'autres sources peuvent compléter l'information
- **Une absence d'information n'est donc pas une négation**
- C'est la raison des réponses « on ne sait pas » des exercices (Rami n'est pas Enseignant ?)

Comment « fermer » le monde : trois techniques

1. Assertion négative

Dire explicitement qu'un fait est faux.

```
<owl:NegativePropertyAssertion>
  <owl:sourceIndividual
     rdf:resource="#Rami"/>
  <owl:assertionProperty
     rdf:resource="#suit"/>
  <owl:targetIndividual
     rdf:resource="#Reseaux"/>
</owl:NegativePropertyAssertion>
```

2. Cardinalité exacte

Fixer le nombre de valeurs, puis déclarer les individus distincts.

Rami suit exactement 2 cours (= 2) et on cite Algo et BD, avec AllDifferent.

Un troisième cours rend la base incohérente.

3. Énumération et ∀

Fermer la liste des valeurs possibles.

owl:oneOf : JourWeekend = {Samedi, Dimanche}.

allValuesFrom : « ne suit que des cours de la liste L ».

06

Raisonnement et pratique

Raisonneurs, profils OWL 2, Protégé, exercices

Les raisonneurs OWL : tâches et outils

Tâche	Question posée	Exemple
Cohérence	L'ontologie contient-elle une contradiction ?	Sami Étudiant + Enseignant (disjoints)
Satisfiabilité	Une classe peut-elle avoir des membres ?	Classe $\sqsubseteq$ A et $\sqsubseteq$ ¬A : insatisfiable
Classification	Quelle est la hiérarchie complète des classes ?	Mère $\sqsubseteq$ Parent déduit
Réalisation	De quelles classes un individu est-il membre ?	Karim type Parent

HermiT, Pellet

Raisonneurs complets OWL 2 DL, intégrés à Protégé.

FaCT++

Raisonneur DL rapide, écrit en C++.

ELK

Très rapide, limité au profil OWL 2 EL (grandes ontologies).

Exemple pas à pas : un raisonnement complet

```
Axiomes :
 (1) aPourEnfant  inverseOf  aPourParent
 (2) aPourEnfant  domain Personne ; range Personne
 (3) Parent ≡ ∃ aPourEnfant . Personne
 (4) Parent  disjointWith  Etudiant
Faits :
 Karim aPourEnfant Sami
 Karim type Etudiant
```

Chaîne de déductions

```
Étape 1 (inverseOf)  : Sami aPourParent Karim
Étape 2 (domain)     : Karim type Personne
Étape 3 (range)      : Sami type Personne
Étape 4 (équivalence): Karim type Parent
Étape 5 (disjoint)   : Karim Parent ET Etudiant
=> ONTOLOGIE INCOHÉRENTE
```

Chaque étape applique un seul axiome. Le raisonneur enchaîne ces étapes jusqu'à épuisement, ou jusqu'à la contradiction.

RAISONNEMENT

Les profils d'OWL 2

Profil	Idée	Usage typique
OWL 2 DL	Expressivité maximale, décidable	Ontologies générales, Protégé
OWL 2 EL	∃, intersection ; pas de ∀ ni complément	Grandes ontologies biomédicales
OWL 2 QL	Interrogation réécrite en SQL	Accès à de grandes bases relationnelles
OWL 2 RL	Règles, raisonnement par règles	Applications à base de règles, triplestores
OWL 2 Full	Aucune restriction, indécidable	Rarement utilisé en pratique

Plus un profil est expressif, plus le raisonnement est coûteux. On choisit le plus petit profil qui suffit au besoin.

Pratique avec Protégé

1. Classes

Onglet Classes : créer Personne, Etudiant, Enseignant, Cours ; déclarer les disjonctions.

2. Propriétés

Object / Data properties : domain, range, caractéristiques (functional, transitive...).

3. Restrictions

Class expression editor : aPourEnfant some Personne, mange only Vegetal.

4. Individus

Onglet Individuals : créer Karim, Sami, ajouter types et relations.

5. Raisonner

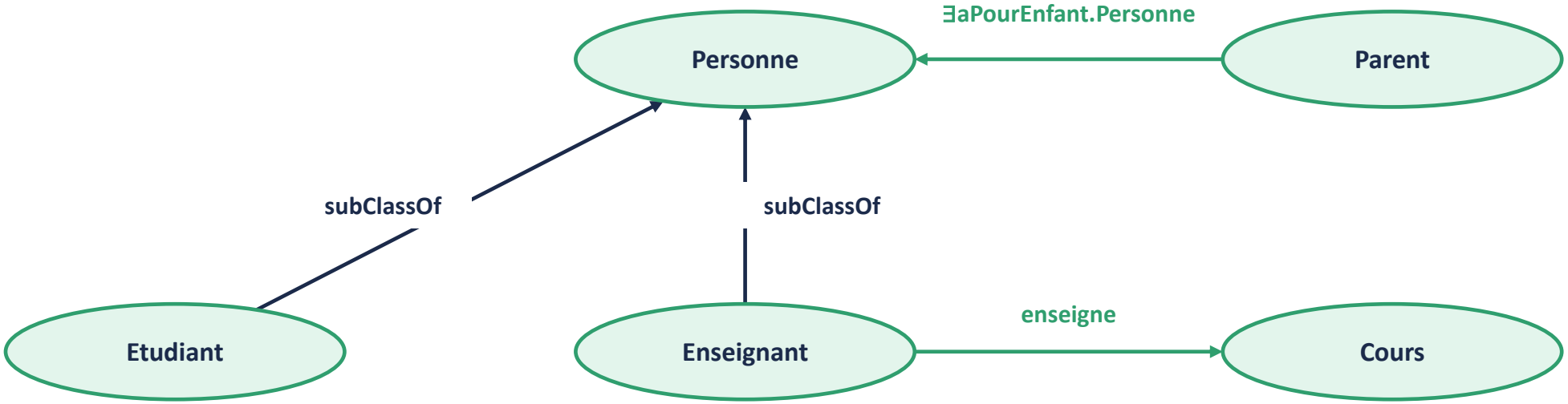
Menu Reasoner : choisir HermiT, Start reasoner.

6. Lire

Déductions en jaune ; classes incohérentes en rouge ; Explain pour justifier.

Les ontologies s'enregistrent en RDF/XML ou Turtle (.owl).

Fil rouge : l'ontologie Université (schéma)



Etudiant et Enseignant sont disjoints. Parent ≡ ∃ aPourEnfant.Personne. enseigne a pour inverse estEnseignéPar. Cours a exactement 1 responsable.

EXEMPLE COMPLET

Fil rouge : le code OWL (extrait)

```
<owl:Class rdf:ID="Personne"/>
<owl:Class rdf:ID="Etudiant">
  <rdfs:subClassOf rdf:resource="#Personne"/>
  <owl:disjointWith rdf:resource="#Enseignant"/>
</owl:Class>
<owl:ObjectProperty rdf:ID="enseigne">
  <rdfs:domain rdf:resource="#Enseignant"/>
  <rdfs:range  rdf:resource="#Cours"/>
  <owl:inverseOf rdf:resource="#estEnseignePar"/>
</owl:ObjectProperty>
<owl:Class rdf:ID="Parent">
  <owl:equivalentClass><owl:Restriction>
    <owl:onProperty rdf:resource="#aPourEnfant"/>
    <owl:someValuesFrom rdf:resource="#Personne"/>
  </owl:Restriction></owl:equivalentClass>
</owl:Class>
```

Individus et déductions

```
Si Karim enseigne Algo :
  Karim type Enseignant (domain)
  Algo type Cours (range)
  Algo estEnseignePar Karim (inverse)
  Karim type Personne (subClassOf)
```

SYNTHÈSE

RDF, RDFS et OWL : comparaison

	RDF	RDFS	OWL
Rôle	Décrire des faits	Vocabulaire simple	Ontologies riches
Structure	Triplets	Classes, propriétés	Restrictions, caractéristiques
Négation / disjonction	Non	Non	Oui (disjointWith, complementOf)
Cardinalités	Non	Non	Oui
Inférence	Aucune	Limitée (type, hiérarchies)	Riche, avec détection d'incohérences
Équivalence / identité	Non	Non	equivalentClass, sameAs

SYNTHÈSE

Erreurs fréquentes

Confondre subClassOf et equivalentClass

subClassOf donne une condition nécessaire seulement : pas de classement automatique.

Domain / range = contrainte

Comme en RDFS, ils servent à déduire un type, pas à rejeter une donnée.

Oublier la disjonction

Sans disjointWith, Étudiant et Enseignant peuvent se recouvrir : aucune incohérence.

Raisonner en monde fermé

L'absence d'un fait ne le rend pas faux. Utiliser des assertions négatives ou des cardinalités.

SYNTHÈSE

Aide-mémoire OWL

Besoin	Élément OWL
Deux classes identiques / sans membre commun	equivalentClass / disjointWith
Combiner des classes	intersectionOf, unionOf, complementOf, oneOf
Type de propriété	ObjectProperty, DatatypeProperty
Caractéristiques	Functional, InverseFunctional, Transitive, Symmetric
Relier des propriétés	inverseOf, equivalentProperty, propertyChainAxiom
Restrictions	someValuesFrom, allValuesFrom, hasValue, cardinality
Identité des individus	sameAs, differentFrom, AllDifferent

07

Exercices

Applications avec corrigés

EXERCICES

Exercices d'application (1/2)

Exercice 1

Écrire en OWL : Mère ≡ Femme ∩ Parent. Karim est Femme et Parent : quelle déduction ?

Exercice 2

estMariéÀ est symétrique. Ali estMariéÀ Sana. Quelle déduction ? Et si on la déclare transitive ?

Exercice 3

Définir Tunisois ≡ habiteEn = Tunis. Mohamed habiteEn Tunis. Que déduit le raisonneur ?

Exercices d'application (2/2)

Exercice 4

Cours a exactement 1 responsable. C1 a deux responsables Karim et Leila. Incohérent ? Dans quel cas ?

Exercice 5

Végétarien $\sqsubseteq \forall$ mange.Végétal. Marc est Végétarien et mange Poulet. Que déduit-on ?

Exercice 6

Rami suit Algo. Peut-on conclure qu'il ne suit pas Réseaux ? Comment l'exprimer ?

EXERCICES

Corrigés (1/2)

Exercice 1

Mère owl:equivalentClass [intersectionOf (Femme Parent)]. Karim type Mère (condition nécessaire et suffisante).

Exercice 2

Symétrique : Sana estMariéÀ Ali. Transitive : Ali estMariéÀ Sana et Sana estMariéÀ Ali donnent Ali estMariéÀ Ali (déduction absurde, éviter).

Exercice 3

Mohamed type Tunisois, par la restriction hasValue (équivalence).

Corrigés (2/2)

Exercice 4

Pas d'incohérence tant qu'on ne dit pas Karim ≠ Leila : le raisonneur déduit Karim sameAs Leila. Avec differentFrom : incohérent.

Exercice 5

Poulet type Végétal (allValuesFrom). Incohérent seulement si Poulet est déclaré disjoint de Végétal.

Exercice 6

Non : monde ouvert. Il faut une NegativePropertyAssertion (Rami suit Réseaux) ou une cardinalité exacte.

À retenir

- OWL étend RDFS : classes complexes, restrictions, caractéristiques de propriétés.
- equivalentClass classe automatiquement ; subClassOf pose une condition nécessaire.
- Pas de nom unique et monde ouvert : l'absence d'un fait n'est pas sa négation.
- Le raisonneur déduit des faits et détecte les incohérences.
- Choisir le profil OWL 2 adapté au besoin.